



(/rol/app/)

SL

Menu

Korean

검색

Seunghe...

Sites

홈 (/rol/app/) (home) > Managing Traffic with OpenShift Service Mesh > Chapter 1: Managing Traffic with OpenShift Service Mesh

Managing Traffic with OpenShift Service Mesh

Lesson

Lab Environment

Guided Exercise: Fault Injection with OpenShift Service Mesh

Apply delay and abort faults to ingress and service-to-service traffic, then observe fault propagation and cascading effects.

Outcomes

- Use Red Hat OpenShift Service Mesh (OSSM) to simulate network failures.
- Inject network faults into direct service requests.
- Inject network faults into service-to-service requests.
- Validate OSSM behavior under network failures.

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshtraffic-chaos
```

Also, run the following command to update your `$PATH` variable and make the `traffic_gen.py` command available immediately. Run it once after creating a new environment.

```
[student@workstation ~]$ source ~/.bashrc
```

The `lab start` command performs the following activities:

- Creates the `meshtraffic-chaos` namespace.
- Adds the `meshtraffic-chaos` namespace to the service mesh.
- Deploys the ratings and three versions of the reviews application in the `meshtraffic-chaos` namespace.
- Configures the `traffic_gen.py` script to generate traffic.
- Creates initial gateway and virtual service OSSM resources for the reviews service.

Instructions

1. Verify the initial status of the service mesh.
 - 1.1. In a new terminal window, log in to the OpenShift cluster as the developer user with the developer password, then switch to the `meshtraffic-chaos` project:

```
[student@workstation ~]$ oc login -u developer \
-p developer https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation ~]$ oc project meshtraffic-chaos
...output omitted...
```

- 1.2. Verify that the applications are running in the `meshtraffic-chaos` namespace.



```
[student@workstation ~]$ oc get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ratings-v1-...	2/2	Running	0	5m
reviews-v1-...	2/2	Running	0	5m
reviews-v2-...	2/2	Running	0	5m
reviews-v3-...	2/2	Running	0	5m

All pods should show 2/2 containers ready, indicating that Istio injected the Envoy sidecar proxy.

1.3. Verify that the required gateway and virtual service OSSM resources exist in the project.

```
[student@workstation ~]$ oc get \
gateways.networking.istio.io,\
virtualservices.networking.istio.io
```

NAME	AGE
gateway.networking.istio.io/reviews-gateway	2m

NAME	GATEWAYS	HOSTS	AGE
virtualservice.networking.istio.io/reviews-vs	["reviews-gateway"]	["*"]	2m

1.4. Change to the exercise directory.

```
[student@workstation ~]$ cd ~/course/labs/meshtraffic-chaos
[student@workstation meshtraffic-chaos]$
```

1.5. Verify external access to the mesh by using the traffic_gen.py script.

```
[student@workstation meshtraffic-chaos]$ traffic_gen.py finite.yaml
🚀 Finite mode: 20 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/20] ✅ HTTP 200 -- No stars (10.5ms)
[2/20] ✅ HTTP 200 -- No stars (7.4ms)
[3/20] ✅ HTTP 200 -- Ratings: ✅ black (20.6ms)
[4/20] ✅ HTTP 200 -- No stars (6.8ms)
[5/20] ✅ HTTP 200 -- No stars (23.6ms)
```

...output omitted...

📊 Traffic Statistics

Total Request	Success Rate	Average	P50	P95
20	100.0% (20/20)	18.4ms	14.6ms	93.5ms

📈 Response Distribution

Response	Count	Percentage
No stars	8	40.0%
Ratings: ✅ black	7	35.0%
Ratings: ✅ red	5	25.0%

Your values might differ.

Notice that all requests succeed with balanced traffic distribution. The baseline average response time is approximately 18ms. This provides a reference for observing the impact of fault injection in subsequent steps.

NOTE

Remember to run the source `~/bashrc` command to update your `$PATH` variable and make the `traffic_gen.py` script available.

```
[student@workstation ~]$ source ~/.bashrc
```

2. Inject a fixed delay fault into the `reviews-vs` virtual service to test direct service requests.

In this step, you inject a 2-second delay into 50% of `reviews` service requests. This simulates external clients experiencing network latency when accessing the service.

Testing delays in direct access enables you to:

- Verify client timeout settings and retry logic
- Test how external users experience slow service responses
- Identify if clients handle latency gracefully or fail abruptly
- Validate SLA compliance under degraded network conditions

2.1. Review the `reviews-vs-delay.yaml` file that configures a delay for the `reviews` service.

- Configures fault injection with a fixed delay at the ingress gateway.
- Applies a 2-second delay to 50% of incoming requests.
- Maintains normal routing to all three `reviews` service versions.

```
[student@workstation meshtraffic-chaos]$ cat reviews-vs-delay.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-vs
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    fault: ❶
      delay: ❷
        percentage:
          value: 50 ❸
        fixedDelay: 2s ❹
    route:
    - destination:
        host: reviews
        port:
          number: 9080
```

- ❶ Configures fault injection for chaos engineering.
- ❷ Specifies latency injection.
- ❸ Controls the percentage of requests receiving the delay
- ❹ Sets the delay duration to 2 seconds.

2.2. Apply the delay configuration to the `reviews` virtual service.

```
[student@workstation meshtraffic-chaos]$ oc replace -f reviews-vs-delay.yaml
virtualservice.networking.istio.io/reviews-vs replaced
```

2.3. Generate traffic to verify the delay.

```
[student@workstation meshtraffic-chaos]$ traffic_gen.py finite.yaml
Finite mode: 20 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/20] ✓ HTTP 200 -- No stars (2047.7ms)
[2/20] ✓ HTTP 200 -- No stars (6.9ms)
[3/20] ✓ HTTP 200 -- No stars (2013.7ms)
[4/20] ✓ HTTP 200 -- No stars (6.9ms)
[5/20] ✓ HTTP 200 -- Ratings: ✓ red (2173.2ms)
[6/20] ✓ HTTP 200 -- Ratings: ✓ red (2040.1ms)
[7/20] ✓ HTTP 200 -- No stars (8.0ms)
[8/20] ✓ HTTP 200 -- Ratings: ✓ red (17.3ms)
[9/20] ✓ HTTP 200 -- No stars (6.5ms)
[10/20] ✓ HTTP 200 -- Ratings: ✓ black (200.2ms)

...output omitted...

Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
20	100.0% (20/20)	837.6ms	81.5ms	2173.2ms

```
...output omitted...
```

Your values might differ.

Notice that approximately 50% of requests show response times around 2 seconds, whereas the other 50% respond normally in milliseconds. All requests still succeed. The average response time of approximately 1 second reflects the 50/50 distribution.

3. Inject an abort fault into the reviews-vs virtual service to test direct service requests.

In this step, you inject HTTP 503 errors into 50% of requests. This simulates service failures that external clients might experience.

Testing aborts in direct access enables you to:

- Verify client error handling and fallback mechanisms
- Test how external users experience service outages
- Identify if clients retry failed requests appropriately
- Validate circuit breaker behavior under failure conditions

3.1. Review the reviews-vs-abort.yaml file that configures aborts for the reviews service.

- Configures fault injection with HTTP abort errors at the reviews-vs virtual service.
- Applies HTTP 503 (Service Unavailable) errors to 50% of incoming requests.
- Maintains normal routing to all three reviews service versions.

```
[student@workstation meshtraffic-chaos]$ cat reviews-vs-abort.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-vs
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    fault:
      abort: ❶
        percentage:
          value: 50.0
        httpStatus: 503 ❷
    route:
    - destination:
        host: reviews
        port:
          number: 9080
```

❶ Specifies abort (error) injection.

❷ Sets the HTTP status code to 503 (Service Unavailable).

3.2. Apply the abort configuration to the reviews-vs virtual service.

```
[student@workstation meshtraffic-chaos]$ oc replace -f reviews-vs-abort.yaml
virtualservice.networking.istio.io/reviews-vs replaced
```

3.3. Generate traffic to verify that HTTP 503 errors are returned for 50% of the requests.

```
[student@workstation meshtraffic-chaos]$ traffic_gen.py finite.yaml
Finite mode: 20 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/20] ✓ HTTP 200 -- Ratings: ✓ black (171.2ms)
[2/20] ✗ HTTP 503 -- (invalid-json) (3.7ms)
[3/20] ✓ HTTP 200 -- Ratings: ✓ black (15.0ms)
[4/20] ✗ HTTP 503 -- (invalid-json) (3.7ms)
[5/20] ✓ HTTP 200 -- Ratings: ✓ red (204.5ms)
[6/20] ✗ HTTP 503 -- (invalid-json) (2.8ms)

...output omitted...

Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
20	55.0% (11/20)	28.8ms	8.4ms	204.5ms

```
...output omitted...
```

Your values might differ.

Notice that approximately 50% of requests fail with HTTP 503 errors, whereas the other 50% succeed normally. Failed requests respond quickly because they are rejected immediately.

3.4. In the workstation machine, open the OpenShift web console in your browser.

Go to <https://console-openshift-console.apps.ocp4.example.com>, click `htpasswd_provider` and use `developer` as the user and the password. If prompted about the guided tour, then click `Skip tour`.

3.5. In the Administrator perspective in the OpenShift console click `Service Mesh` → `Traffic Graph`.

3.6. Add the meshtraffic-chaos namespace to the graph by selecting it in `Select Namespaces`.

Observe the fault injection behavior in the traffic graph:

- A red arrow connects istio-ingressgateway to the reviews service, indicating HTTP errors.
- The Current graph summary panel on the right shows approximately 50% HTTP 5xx errors.
- The error rate matches the 50% abort configuration in the virtual service.
- All three reviews service versions receive traffic, but 50% of requests fail before reaching the application code.

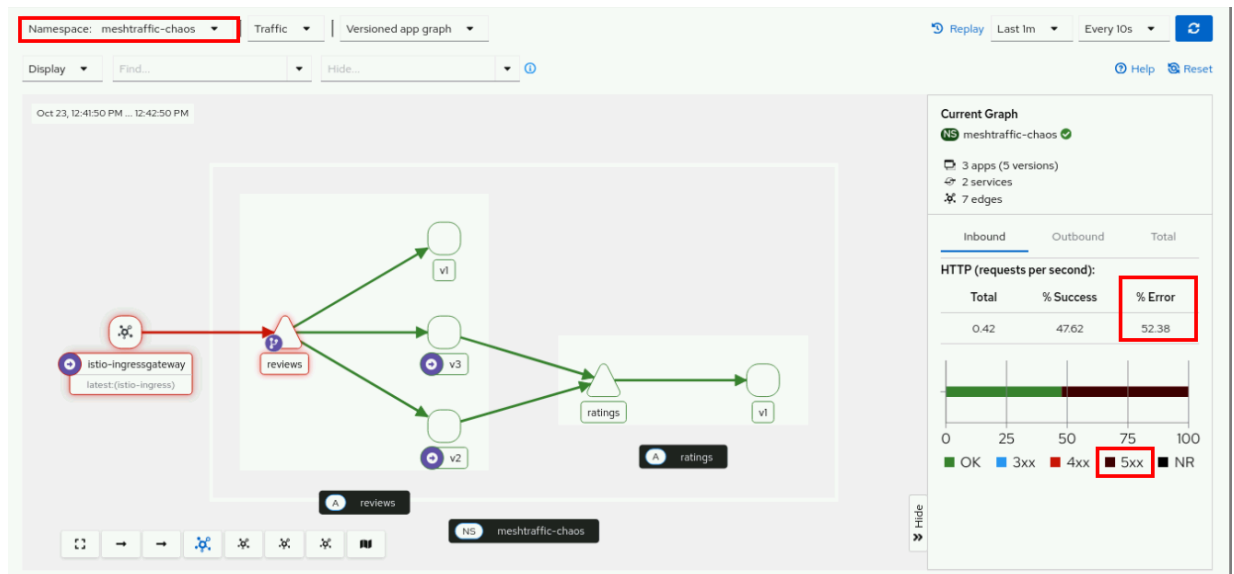


Figure 1.7: Traffic graph showing 50% HTTP 503 errors from abort fault injection

Leave the graphic page open. You return to this graph throughout the exercise to observe how fault injection affects network activity.

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Generate more traffic by refreshing the browser page or re-running the `traffic_gen.py` script.

4. Inject a delay fault into the ratings virtual service to test service-to-service requests.

In previous steps, you tested faults in direct requests. Now you test service-to-service faults.

In this step, you inject a delay into 50% of requests to the ratings service. This delay occurs when reviews calls ratings, demonstrating how faults propagate through service dependencies.

Testing delays in service-to-service communication enables you to:

- Observe how delays propagate through service dependencies
- Test timeout configurations between internal services
- Identify cascading latency in microservice architectures
- Validate how dependent services handle slow upstream services

4.1. Re-establish the reviews-vs virtual service without faults.

```
[student@workstation meshtraffic-chaos]$ oc replace -f reviews-vs.yaml
virtualservice.networking.istio.io/reviews-vs replaced
```

4.2. Review the ratings-vs-delay.yaml file that adds a delay to the ratings service.

- Configures fault injection with a delay.
- Applies the delay to 50% of requests.

```
[student@workstation meshtraffic-chaos]$ cat ratings-vs-delay.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings-vs
spec:
  hosts:
  - ratings.meshtraffic-chaos.svc.cluster.local
  http:
  - match:
    - uri:
        prefix: /ratings
    fault: ❶
      delay: ❷
        percentage:
          value: 50 ❸
        fixedDelay: 2s ❹
    route:
    - destination:
        host: ratings
        port:
          number: 9080
```

- ❶ Configures fault injection for chaos engineering.
- ❷ Specifies delay injection.
- ❸ Controls the percentage of requests receiving the delay.
- ❹ Sets the delay duration to 2 seconds.

4.3. Create the ratings-vs virtual service for the ratings service to add the delay.

```
[student@workstation meshtraffic-chaos]$ oc create -f ratings-vs-delay.yaml
virtualservice.networking.istio.io/ratings-vs created
```

4.4. Use the traffic_gen.py script to verify the delay.

```
[student@workstation meshtraffic-chaos]$ traffic_gen.py finite.yaml
traffic_gen.py finite.yaml
🔗 Finite mode: 20 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/20] ✅ HTTP 200 -- Ratings: ✅ red (2099.4ms)
[2/20] ✅ HTTP 200 -- Ratings: ✅ black (124.9ms)
[3/20] ✅ HTTP 200 -- No stars (44.3ms)
[4/20] ✅ HTTP 200 -- Ratings: ✅ red (2015.0ms)
[5/20] ✅ HTTP 200 -- Ratings: ✅ red (16.2ms)
[6/20] ✅ HTTP 200 -- Ratings: ✅ black (29.0ms)
[7/20] ✅ HTTP 200 -- No stars (13.1ms)
[8/20] ✅ HTTP 200 -- Ratings: ✅ black (17.5ms)
[9/20] ✅ HTTP 200 -- Ratings: ✅ red (2030.7ms)
[10/20] ✅ HTTP 200 -- No stars (8.9ms)

...output omitted...

📊 Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
20	100.0% (20/20)	526.7ms	16.7ms	2099.4ms

```

...output omitted...

```

Your values might differ.

Observe how the delay propagates through service dependencies:

- reviews-v1 (No stars): Consistently fast (~10-60ms) because v1 does not call ratings.
- reviews-v2 (black) and reviews-v3 (red): Show mixed response times - fast (~17-19ms) when ratings responds normally, delayed (~2000ms) when the fault is injected into ratings.

- All requests succeed (100%), showing that delays increase latency but do not cause failures.
- 4.5. Go to the **Service Mesh** → **Traffic Graph** graphic at the OpenShift web console in your browser.
 - 4.6. Make sure you add the `meshtraffic-chaos` namespace to the graph by selecting it in **Select Namespaces**.
 - 4.7. In the **Display** combo box, select **Response Time** → **95th Percentile**.

Observe the delay propagation in the traffic graph:

- Only service versions with dependencies on `ratings` exhibit increased latency.
- `reviews-v1` has no connection to `ratings` and maintains normal response times.

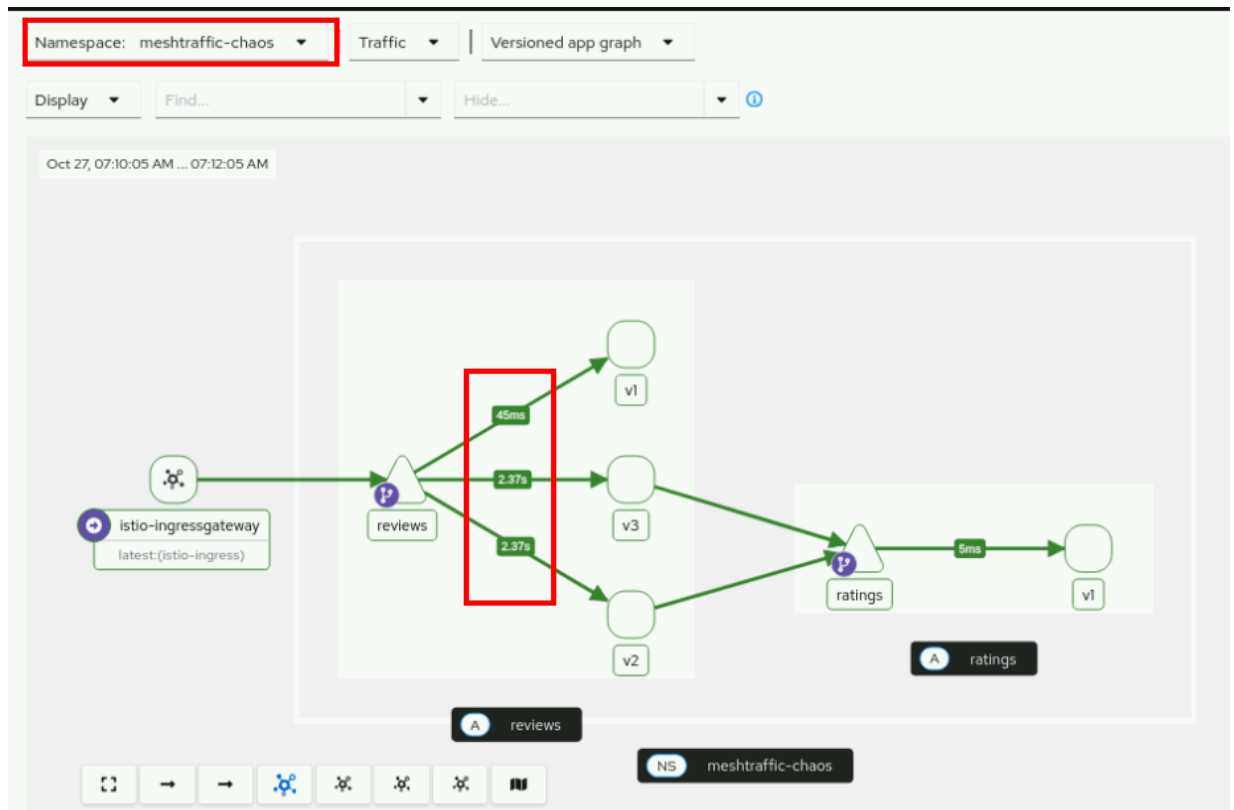


Figure 1.8: Traffic graph showing response time delays in service-to-service communication

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Generate more traffic by refreshing the browser page or re-running the `traffic_gen.py` script.

5. Inject an abort fault into the `ratings` virtual service to test service-to-service requests.

In this step, you inject HTTP 500 errors into 50% of requests to the `ratings` service when `reviews` requests it.

Testing aborts in service-to-service communication enables you to:

- Observe how failures cascade through microservice architectures
- Test error handling between dependent services

- 5.1. Review the `ratings-vs-error.yaml` file, configuring aborts for the `ratings` service.

- Configures fault injection with HTTP abort faults.
- Returns HTTP 500 (Internal Server Error) for 50% of requests.
- Simulates service failures without affecting the actual backend.


```
[student@workstation meshtraffic-chaos]$ cat ratings-vs-error.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings-vs
spec:
  hosts:
  - ratings.meshtraffic-chaos.svc.cluster.local
  http:
  - match:
    - uri:
        prefix: /ratings
    fault:
      abort: ❶
      httpStatus: 500 ❷
      percentage:
        value: 50.0
    route:
    - destination:
        host: ratings
        port:
          number: 9080
```

❶ Specifies HTTP error injection.

❷ Sets the HTTP error code to return (500 = Internal Server Error).

5.2. Update the ratings-vs virtual service.

```
[student@workstation meshtraffic-chaos]$ oc replace -f ratings-vs-error.yaml
virtualservice.networking.istio.io/ratings-vs replaced
```

5.3. Use the traffic_gen.py script to verify how the reviews service handles the intermittent ratings HTTP 500 errors.

```
[student@workstation meshtraffic-chaos]$ traffic_gen.py finite.yaml
Finite mode: 20 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/20] ✓ HTTP 200 -- Ratings: ✓ black (17.2ms)
[2/20] ✓ HTTP 200 -- Ratings: ✓ black (13.4ms)
[3/20] ✓ HTTP 200 -- Ratings: ✓ red (13.3ms)
[4/20] ✓ HTTP 200 -- Ratings: ✗ Ratings service is currently unavailable (12.5ms)
[5/20] ✓ HTTP 200 -- Ratings: ✓ red (13.4ms)
[6/20] ✓ HTTP 200 -- Ratings: ✓ black (22.4ms)
[7/20] ✓ HTTP 200 -- No stars (7.4ms)
[8/20] ✓ HTTP 200 -- Ratings: ✗ Ratings service is currently unavailable (12.3ms)
[9/20] ✓ HTTP 200 -- Ratings: ✗ Ratings service is currently unavailable (15.9ms)
[10/20] ✓ HTTP 200 -- No stars (7.8ms)
```

...output omitted...

Traffic Statistics

Total Request	Success Rate	Average	P50	P95
20	100.0% (20/20)	13.2ms	13.1ms	23.3ms

Response Distribution

Response	Count	Percentage
Ratings: ✗ Ratings service is c...	6	30.0%
No stars	6	30.0%
Ratings: ✓ black	4	20.0%
Ratings: ✓ red	4	20.0%

Notice that although the ratings service returns HTTP 500 errors, the reviews service handles these failures gracefully and returns HTTP 200 responses to clients.

- 5.4. Go to the **Service Mesh → Traffic Graph** graphic at the OpenShift web console in your browser.
- 5.5. Make sure you add the `meshtraffic-chaos` namespace to the graph by selecting it in **Select Namespaces**.
Observe the error propagation in the traffic graph image:

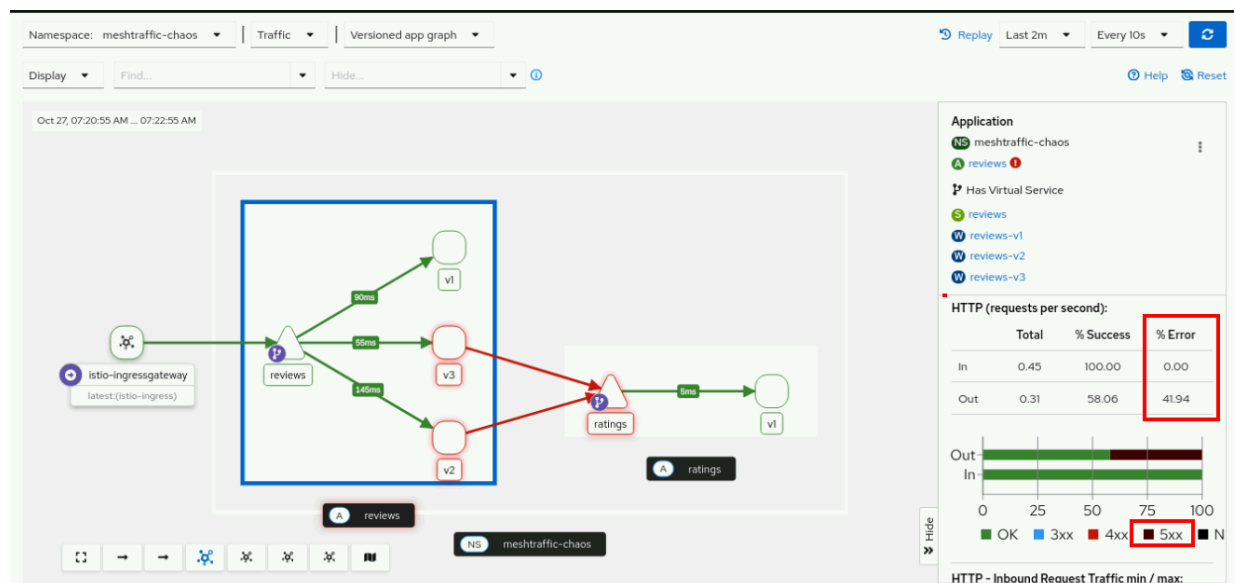


Figure 1.9: Traffic graph showing errors in service-to-service communication

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Generate more traffic by refreshing the browser page or re-running the `traffic_gen.py` script.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshtraffic-chaos
```

Revision: ad0008l-3.1-ed4-20260310-8ae9760



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책 (<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침 (<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정